

تمرین عملی سری اول

مبانی هوش مصنوعی

مبحث: پیاده سازی الگوریتم های جستجو

تاریخ انتشار: ۰۲/۱۴

نکات تمرین:

دانشجویان عزیز موظف هستند با توجه به آنچه در کلاس آموخته اند و همچنین در صورت نیاز، مطالعه کتاب و مرجع به سوالات پاسخ دهند.

گزارش و کد های خواسته شده را به صورت یک فایل فشرده درآورده و نام فایل به شرح زیر باشد:

HW1_P_<student name>_<studentID>

پرسش و پاسخ های مربوط به تمرین را حتما در گروه ذکر کنید تا اگر این موارد سوال باقی دانشجویان درس هم بود به آن پاسخ داده شود.

به هیچ وجه تمرینی را از دیگران کپی نکنید. در صورت مشاهده تقلب و کپی در تمرینات، نمره هر دو طرف صفر در نظر گرفته می شود.

در این سری از تمرین در صورت استفاده از هرگونه هوش مصنوعی و چت بات حتما در گزارش خود دلیل و نوع استفاده را ذکر کنید .

تحويل تمرین تنها از طریق کوئرا صورت می گیرد.

تمرین عملی جستجو

یک ربات در یک هزارتوی بزرگ گیر کرده است و شما باید به او با شرایط زیر کمک کنید تا از این هزارتو خارج شود:

- بعضی موانع سر جای خودشان ثابت می‌مانند.
 - برخی دیگر از موانع در طول زمان حرکت می‌کنند و ممکن است برای مسیر اولیه اختلال به وجود بیاورند.
 - بعضی سلول‌ها هزینه بیشتری دارند و باید از این جلوگیری کرد.
- شما باید برای این ربات الگوریتم‌های جستجوی مختلف را پیاده‌سازی کرده و در نهایت تحلیل کنید که به صورت میانگین کدام الگوریتم به جواب سریع‌تری می‌رسد و بهترین مسیر را بدون هزینه زیاد پیدا می‌کند. دقت کنید که الگوریتم‌ها عبارتند از:

- الگوریتم BFS
- الگوریتم DFS
- الگوریتم UCS
- الگوریتم A*

در نهایت این الگوریتم‌ها را از نظر تعداد حرکت و زمان و تعداد گره‌های گسترش یافته و هزینه کل مسیر مقایسه کنید (به صورت دستی محاسبه کردن و بدون اضافه کردن کد هم این عمل ممکن است و موردی ندارد).

در این تمرین دو فایل `grid_env.py` و `search.py` داده شده و صرفاً باید الگوریتم‌های خواسته شده را در فایل `search.py` پیاده‌سازی کنید.

در نظر داشته باشید برای تحویل تمرین باید یک فایل گزارش آماده کنید که در آن به صورت خلاصه رویکرد، کدهای خود و خروجی و مقایسه خواسته شده را توضیح دهید.

قبل از پیاده‌سازی کد به نکاتی که در ادامه راجع به محیط برنامه و چگونگی اجرا کد گفته شده توجه کنید.

اجرای کد

برای اجرای کد به پکیج pygame نیاز دارید، برای نصب آن از دستور زیر می توانید استفاده کنید:

```
pip install pygame -i https://package-mirror.liara.ir/repository/pypi/simple
--trusted-host package-mirror.liara.ir
```

فایل grid_env.py

این فایل محیط را ایجاد و نمایش می‌دهد. شما لازم نیست این فایل را تغییر دهید.

ویژگی‌های محیط:

- محیط یک grid با اندازه 15×15 است.
- هر خانه می‌تواند یکی از حالت‌های زیر را داشته باشد :
 - خانه عادی با هزینه حرکت 1
 - خانه سخت‌گذر با هزینه حرکت 5
 - مانع ثابت، غیر قابل عبور
 - مانع متحرک، غیر قابل عبور در زمان برنامه‌ریزی مسیر
- نقطه شروع به رنگ سبز نمایش داده می‌شود.
- نقطه هدف به رنگ قرمز نمایش داده می‌شود.
- موانع ثابت به رنگ مشکی نمایش داده می‌شوند.
- خانه‌های سخت‌گذر به رنگ قهوه‌ای نمایش داده می‌شوند.
- موانع متحرک به رنگ آبی نمایش داده می‌شوند.
- موقعیت فعلی عامل با رنگ فیروزه‌ای نمایش داده می‌شود.
- عامل در یک محیط دوبعدی قرار دارد. هر موقعیت به صورت یک زوج مرتب نمایش داده می‌شود:

(row, col)

برای مثال:

```
start = (2, 3)
goal = (10, 12)
```

- عامل می‌تواند در چهار جهت حرکت کند:

بالا، پایین، چپ، راست

- علاوه بر این، تعدادی مانع متحرک نیز وجود دارد که در هر فریم حرکت می‌کنند. در زمان محاسبه مسیر، خانه‌هایی که در آن لحظه شامل مانع متحرک هستند نیز غیرقابل عبور در نظر گرفته می‌شوند.

تفاوت ورودی تابع کمکی neighbors_fn در الگوریتم‌ها:

برای DFS و BFS، تابع neighbors_fn لیستی از همسایه‌ها را برمی‌گرداند:

`[(x1, y1), (x2, y2), ...]`

اما برای UCS و A*، چون هزینه مهم است، تابع neighbors_fn لیستی از زوج‌های زیر را برمی‌گرداند:

`[((x1, y1), cost1), ((x2, y2), cost2), ...]`

بنابراین در پیاده‌سازی UCS و A* باید به این تفاوت توجه کنید.

محیط برنامه

پس از اجرا، برنامه از شما می‌خواهد الگوریتم را انتخاب کنید:

- 1: DFS
- 2: BFS
- 3: UCS
- 4: A* (Manhattan)
- 5: A* (Euclidean)

با وارد کردن عدد مربوطه، عامل مسیر را محاسبه و در محیط گرافیکی حرکت می‌کند.

کلیدهای قابل استفاده:

شروع دوباره با محیط جدید: R

خروج از برنامه: ESC